



AMIRKABIR UNIVERSITY OF TECHNOLOGY

Department of Mathematics and Computer Science

Numerical Methods Toolkit

Computational Projects in Numerical Analysis

Project Scope

Implementation and numerical evaluation of interpolation methods, automated Excel-based data analysis, and numerical integration methods using Python and scientific-computing libraries.

Name: Arad Shafiee

Student ID: 40312029

Course: Mathematical Software

Date: July 2026 (Tir 1405)

Abstract

This report presents a computational study of three fundamental areas of numerical methods: interpolation, data analysis, and numerical integration. The first project compares global polynomial interpolation using the Lagrange formulation with a natural cubic spline constructed from the same set of observations. The second project develops an automated Python workflow for reading algorithm-runtime measurements from an Excel spreadsheet, performing descriptive statistical analysis, generating multiple visualizations, and extending the dataset with a required 700KB observation. The third project investigates numerical approximation of the definite integral $\int_0^1 e^{x^2} dx$ using the composite trapezoidal rule, composite Simpson's rule, and Gauss–Legendre quadrature.

The experiments emphasize not only numerical results but also the mathematical properties of the methods, including interpolation conditions, smoothness, convergence order, empirical runtime behavior, and numerical error. The interpolation experiment demonstrates the distinction between a global high-degree polynomial and a locally defined spline representation. The runtime analysis illustrates how numerical programming can be combined with reproducible data-processing and visualization pipelines. The integration experiment shows the faster convergence of Simpson's rule relative to the trapezoidal rule and the very high accuracy achievable by Gaussian quadrature for a smooth integrand.

All calculations and figures are generated programmatically, providing a reproducible computational workflow. The results are interpreted conservatively, particularly where the available dataset is insufficient to support stronger theoretical conclusions.

Contents

1	Introduction	1
2	Computational Environment and Reproducibility	1
2.1	Software Stack	1
2.2	Reproducible Workflow	2
3	Project 1: Interpolation Methods	3
3.1	Problem Formulation	3
3.2	Lagrange Polynomial Interpolation	3
3.3	Natural Cubic Spline Interpolation	4
3.4	Graphical Results	5
3.5	Discussion	5
4	Project 2: Excel-Based Algorithm Runtime Analysis	6
4.1	Problem Formulation	6
4.2	Original Dataset	6
4.3	Original Bar Chart	6
4.4	Original Line Chart	7
4.5	Original Box Plot	7
4.6	Descriptive Statistics	8
4.7	Mean Runtime of Algorithm 2	8
4.8	Automatic Extension of the Dataset	9
4.9	Updated Bar Chart	9
4.10	Updated Line Chart	10
4.11	Updated Box Plot	10
4.12	Runtime Interpretation	11
5	Project 3: Numerical Integration	12
5.1	Problem Formulation	12
5.2	Reference Value	12
5.3	Composite Trapezoidal Rule	12
5.4	Composite Simpson's Rule	13
5.5	Gauss–Legendre Quadrature	13
5.6	Trapezoidal and Simpson Results	13
5.7	Gaussian Quadrature Results	14
5.8	Graphical Convergence Analysis	14
5.9	Method Comparison	15
6	Cross-Project Discussion	16
6.1	Approximation and Reference Values	16
6.2	Convergence as a Diagnostic Tool	16
6.3	Global Versus Local Approximation	16

6.4	Data Analysis as a Numerical Workflow	16
7	Limitations and Future Work	17
7.1	Interpolation	17
7.2	Runtime Analysis	17
7.3	Numerical Integration	17
8	Conclusion	17
9	Source Code and Repository	19
	References	20

List of Figures

1	Lagrange Polynomial and Natural Cubic Spline Interpolation	5
2	Algorithm Runtime Comparison – Original Dataset	6
3	Algorithm Runtime Trends – Original Dataset	7
4	Runtime Distribution – Original Dataset	8
5	Algorithm Runtime Comparison – Updated Dataset	9
6	Algorithm Runtime Trends – Updated Dataset	10
7	Runtime Distribution – Updated Dataset	10
8	Numerical Integration Comparison and Convergence	15

List of Tables

1	Software Libraries Used in the Project	2
2	Interpolation Dataset	3
3	Natural Cubic Spline Coefficients	4
4	Original Algorithm Runtime Dataset	6
5	Descriptive Statistics for the Original Dataset	8
6	Added 700KB Observation	9
7	Convergence of Trapezoidal and Simpson Methods	13
8	Convergence of Gauss–Legendre Quadrature	14
9	Summary of Numerical Integration Methods	15

1 Introduction

Numerical analysis provides systematic methods for obtaining approximate solutions to mathematical problems that are difficult or impossible to solve directly by symbolic means. Its applications extend across engineering, physics, computer science, statistics, and applied mathematics. Classical numerical techniques include interpolation, numerical differentiation and integration, nonlinear equation solving, linear algebra, and differential-equation methods [1, 2].

The present project focuses on three computational tasks that illustrate different aspects of numerical computation. The first concerns interpolation, where an approximation is constructed from discrete data points. The second concerns data analysis, where computational methods are used to process and visualize measured execution times. The third concerns numerical integration, where a definite integral is approximated through several quadrature techniques.

The main objectives are:

1. To implement Lagrange polynomial interpolation and natural cubic spline interpolation for a common dataset.
2. To compare the mathematical structure and numerical behavior of the two interpolation approaches.
3. To develop a reproducible workflow for reading and analyzing algorithm-runtime data from an Excel spreadsheet.
4. To use descriptive statistics and multiple visualization methods to investigate empirical runtime behavior.
5. To implement and compare the composite trapezoidal rule, composite Simpson's rule, and Gauss–Legendre quadrature.
6. To analyze convergence and numerical error rather than relying solely on a single computed result.
7. To maintain a reproducible computational project in a version-controlled repository.

A central principle throughout the project is that numerical results should be interpreted in relation to the assumptions and limitations of the underlying methods. In particular, empirical observations should not be mistaken for formal proofs of asymptotic complexity or exact analytical solutions.

2 Computational Environment and Reproducibility

2.1 Software Stack

The implementation is developed in Python using standard scientific computing and data-analysis libraries.

Table 1: *Software Libraries Used in the Project*

Library	Main Purpose
NumPy	Numerical arrays and mathematical operations
Pandas	Data loading, transformation, and statistical analysis
Matplotlib	Scientific visualization and PDF/PNG figure generation
SciPy	Numerical integration and reference computation
OpenPyXL	Excel workbook support

Pandas provides the `read_excel` interface used to load the Excel dataset into a DataFrame [3]. Matplotlib’s `savefig` interface is used to export generated figures, including PDF vector graphics suitable for inclusion in a technical report [4].

For the numerical reference value in the integration experiment, SciPy’s `scipy.integrate.quad` function is used. The routine performs adaptive quadrature through the QUADPACK library and returns both an integral estimate and an error estimate [5, 6].

2.2 Reproducible Workflow

The project follows a simple reproducible pipeline:

1. Input data are stored separately from source code.
2. Numerical calculations are performed programmatically.
3. Generated figures and numerical summaries are saved as output artifacts.
4. The report incorporates the generated PDF figures directly.
5. Source code, input data, outputs, and documentation are managed through Git.

This organization makes it possible to rerun the experiments and regenerate the computational results rather than relying on manually entered values.

3 Project 1: Interpolation Methods

3.1 Problem Formulation

Interpolation estimates the value of a function at points located between known observations. Given a set of distinct points (x_i, y_i) , an interpolation function $F(x)$ is required to satisfy

$$F(x_i) = y_i$$

at every supplied node.

The dataset used in this experiment is

Table 2: *Interpolation Dataset*

x	1	2	3	4	5	6
y	1	3	5	8	5	2

Two interpolation approaches are investigated:

- Lagrange polynomial interpolation.
- Natural cubic spline interpolation.

Classical treatments of interpolation discuss the mathematical properties, error behavior, and limitations of polynomial and spline methods [1, 2].

3.2 Lagrange Polynomial Interpolation

For $n + 1$ distinct nodes, the Lagrange interpolation polynomial is

$$P_n(x) = \sum_{i=0}^n y_i L_i(x),$$

where

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}.$$

The basis polynomials satisfy

$$L_i(x_j) = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

Consequently, the resulting polynomial satisfies the interpolation conditions exactly.

For the six observations in Table 2, the polynomial has degree five:

$$P(x) = \frac{7}{40}x^5 - \frac{71}{24}x^4 + \frac{147}{8}x^3 - \frac{1249}{24}x^2 + \frac{1369}{20}x - 31$$

or numerically,

$$P(x) = 0.1750x^5 - 2.9583x^4 + 18.3750x^3 \\ - 52.0417x^2 + 68.4500x - 31.$$

The polynomial therefore satisfies

$$P(1) = 1, \quad P(2) = 3, \quad P(3) = 5, \quad P(4) = 8, \quad P(5) = 5, \quad P(6) = 2.$$

Numerical result. The Lagrange implementation produces a fifth-degree polynomial that passes exactly through all six supplied interpolation nodes.

3.3 Natural Cubic Spline Interpolation

A cubic spline replaces one global polynomial with a collection of cubic polynomials defined on consecutive intervals. For $x \in [x_i, x_{i+1}]$,

$$S_i(x) = a_i(x - x_i)^3 + b_i(x - x_i)^2 + c_i(x - x_i) + d_i.$$

A natural cubic spline satisfies interpolation, continuity of the first derivative, continuity of the second derivative, and the natural boundary conditions

$$S''(x_0) = 0, \quad S''(x_n) = 0.$$

These conditions produce a smooth piecewise polynomial approximation [1, 2].

The resulting coefficients are given in Table 3.

Table 3: *Natural Cubic Spline Coefficients*

Interval	a_i	b_i	c_i	d_i
[1, 2]	-0.1866	0.0000	2.1866	1.0000
[2, 3]	0.9330	-0.5598	1.6268	3.0000
[3, 4]	-2.5455	2.2392	3.3062	5.0000
[4, 5]	2.2488	-5.3971	0.1483	8.0000
[5, 6]	-0.4498	1.3493	-3.8995	5.0000

For example, on the first interval,

$$S_1(x) = -0.1866(x - 1)^3 + 2.1866(x - 1) + 1.$$

On the second interval,

$$S_2(x) = 0.9330(x - 2)^3 - 0.5598(x - 2)^2 + 1.6268(x - 2) + 3.$$

The remaining intervals are defined by the coefficients in Table 3.

3.4 Graphical Results

Figure 1 presents the generated comparison between the Lagrange polynomial and the natural cubic spline.

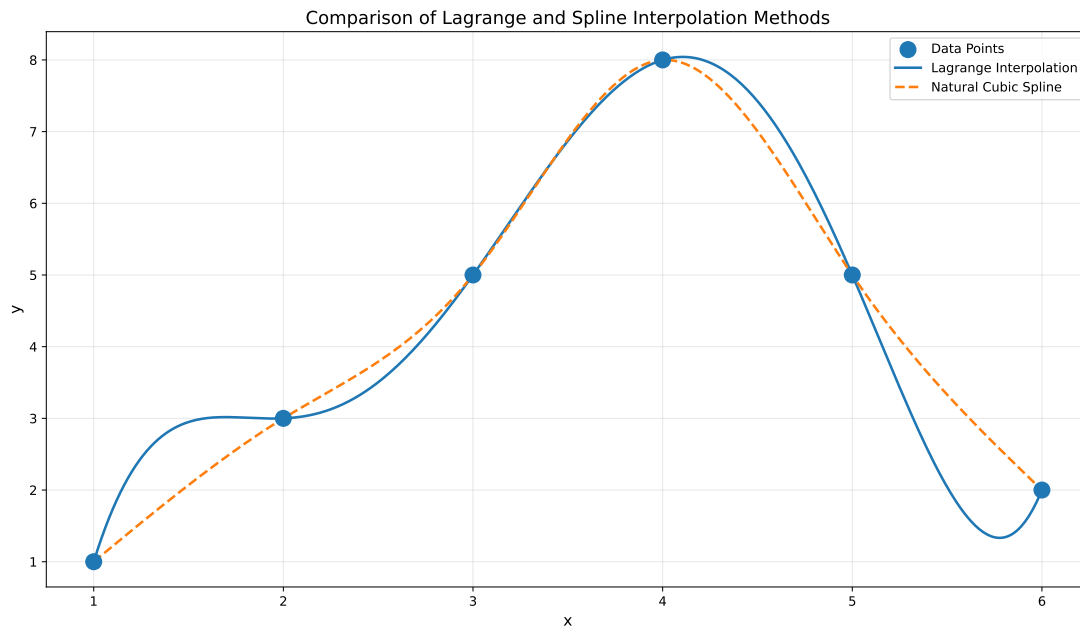


Figure 1: *Lagrange Polynomial and Natural Cubic Spline Interpolation*

The figure provides a visual representation of the global polynomial behavior and the piecewise spline approximation.

3.5 Discussion

The two methods satisfy the same interpolation constraints but use different mathematical constructions.

The Lagrange method produces one polynomial over the entire interval. This global structure is convenient for a small number of nodes, but increasing the polynomial degree can introduce oscillatory behavior and can make the approximation sensitive to the node distribution.

The spline method instead uses local cubic pieces. The smoothness constraints connect the individual pieces, while the low degree of each piece limits the amount of global variation. This local structure is one reason cubic splines are widely used for practical interpolation [1].

It is important to note that the present experiment contains only six points. Therefore, the figure demonstrates the behavior of the two methods for this particular dataset and should not be interpreted as a universal benchmark.

4 Project 2: Excel-Based Algorithm Runtime Analysis

4.1 Problem Formulation

The second project analyzes execution-time measurements for three algorithms at different input sizes. The original dataset contains six observations between 100KB and 600KB.

The computational workflow reads the Excel workbook, validates the columns, performs numerical conversion and statistical analysis, generates three visualization types, and finally extends the dataset with a required 700KB observation.

4.2 Original Dataset

Table 4: *Original Algorithm Runtime Dataset*

Data Size	Algorithm 1	Algorithm 2	Algorithm 3
100KB	50	200	100
200KB	55	220	200
300KB	60	240	300
400KB	65	260	400
500KB	70	280	500
600KB	75	300	600

The dataset contains only a small number of observations. Consequently, the purpose of the analysis is descriptive rather than a formal determination of asymptotic algorithmic complexity.

4.3 Original Bar Chart

The bar chart provides a direct comparison of execution times.

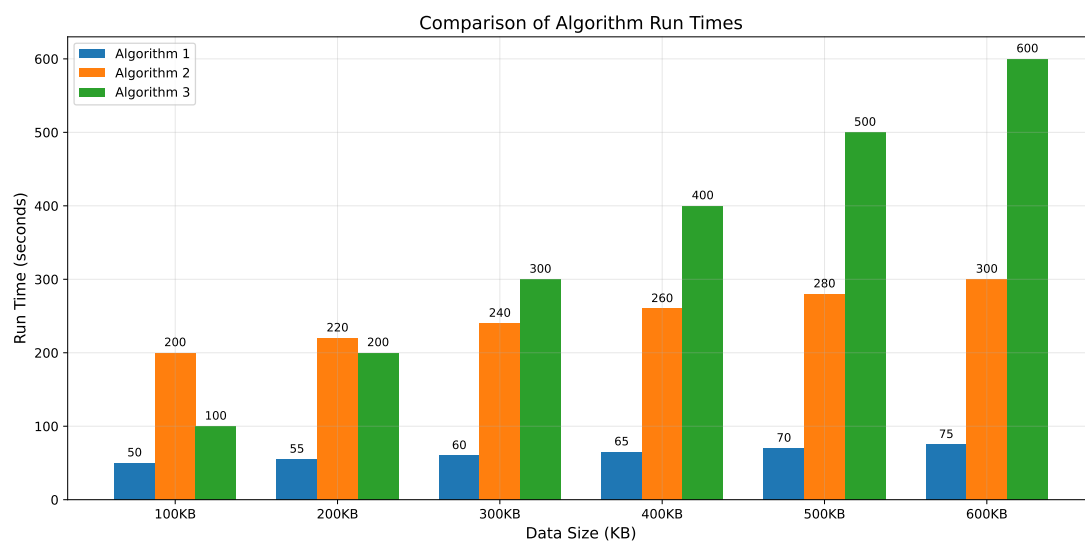


Figure 2: *Algorithm Runtime Comparison – Original Dataset*

Algorithm 1 has the smallest runtime for every supplied input size, whereas Algorithm 3

has the largest runtime at the larger input sizes.

4.4 Original Line Chart

The line chart emphasizes how runtime changes with input size.

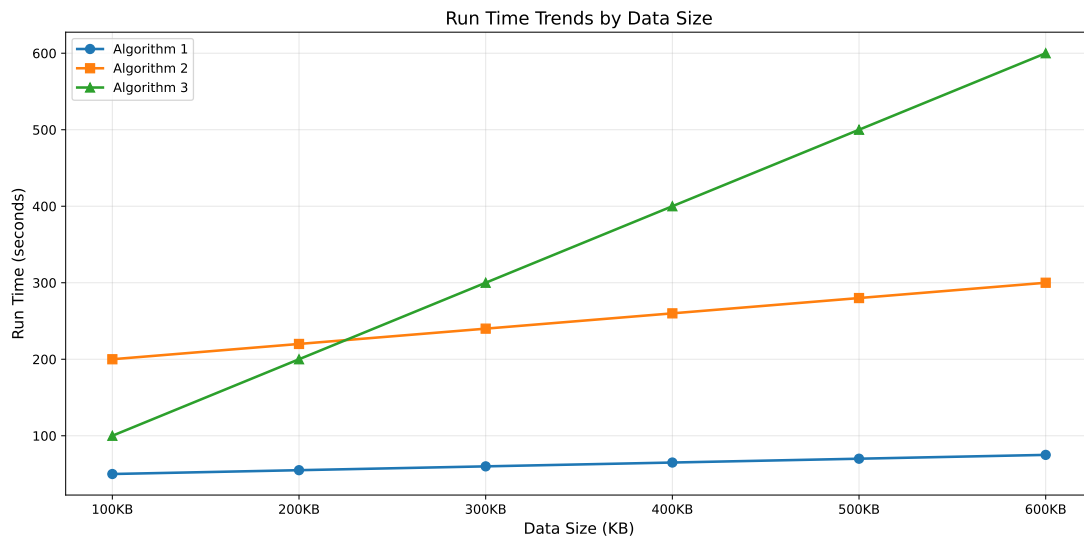


Figure 3: *Algorithm Runtime Trends – Original Dataset*

All three algorithms exhibit increasing runtime as the input size increases. Over the observed range, the measurements are approximately linear.

It would not be statistically or theoretically justified to infer an exact asymptotic complexity class from these seven or fewer observations alone.

4.5 Original Box Plot

The box plot summarizes the distribution of the observed runtimes.

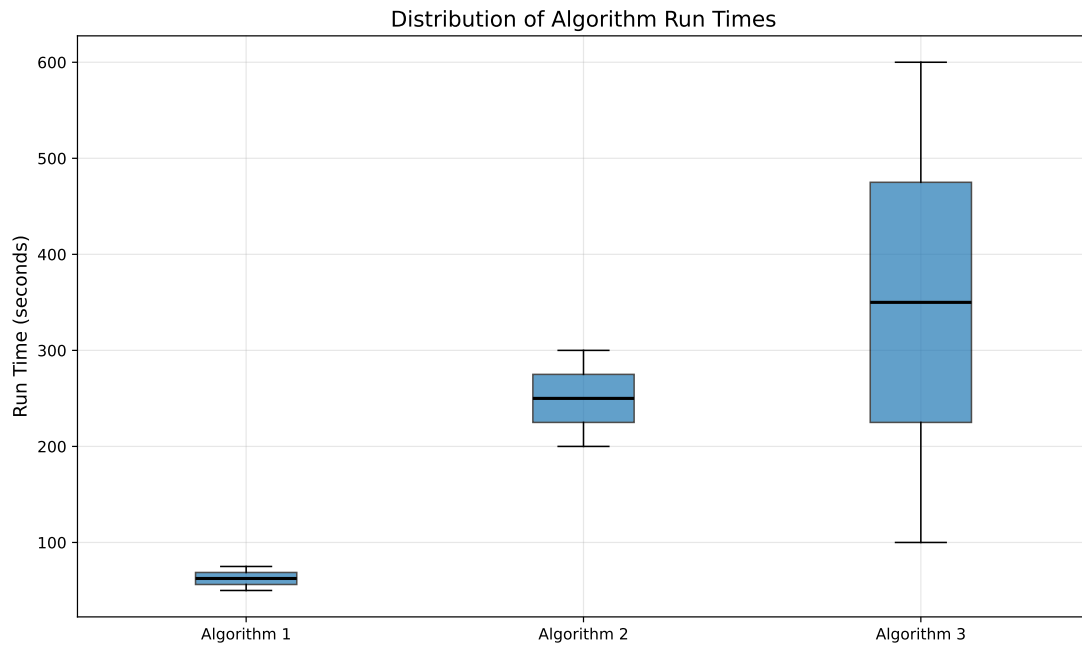


Figure 4: *Runtime Distribution – Original Dataset*

Algorithm 3 exhibits the largest spread, which is consistent with its larger range and standard deviation.

4.6 Descriptive Statistics

The calculated statistics are presented in Table 5.

Table 5: *Descriptive Statistics for the Original Dataset*

Statistic	Algorithm 1	Algorithm 2	Algorithm 3
Mean	62.50	250.00	350.00
Median	62.50	250.00	350.00
Sample Std. Dev.	9.35	37.42	187.08
Minimum	50.00	200.00	100.00
Maximum	75.00	300.00	600.00
Range	25.00	100.00	500.00

The sample standard deviation is

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}.$$

Algorithm 3 therefore exhibits considerably greater variability within the observed sample.

4.7 Mean Runtime of Algorithm 2

The required mean for Algorithm 2 over the original 100–600KB range is

$$\bar{x} = \frac{200 + 220 + 240 + 260 + 280 + 300}{6} = 250.$$

Hence,

$$\bar{x}_{\text{Algorithm 2}} = 250 \text{ seconds}$$

This calculation intentionally excludes the later 700KB observation.

4.8 Automatic Extension of the Dataset

The project requires the addition of the following observation:

Table 6: *Added 700KB Observation*

Data Size	Algorithm 1	Algorithm 2	Algorithm 3
700KB	80	320	700

The implementation checks whether a 700KB observation already exists before inserting it. This prevents duplicate observations when the program is executed repeatedly.

4.9 Updated Bar Chart

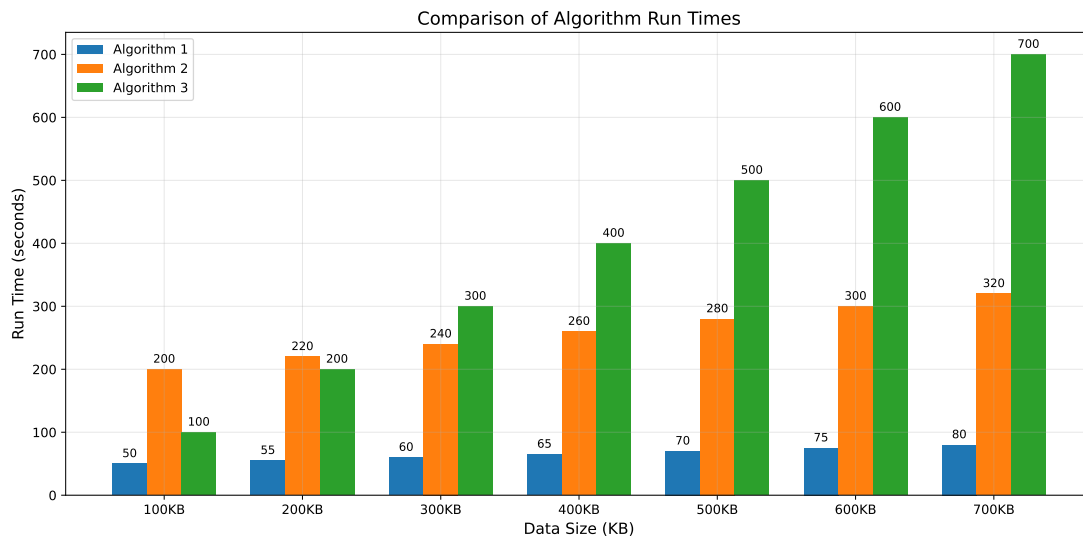


Figure 5: *Algorithm Runtime Comparison – Updated Dataset*

The new observation extends the existing runtime trend without changing the previous measurements.

4.10 Updated Line Chart

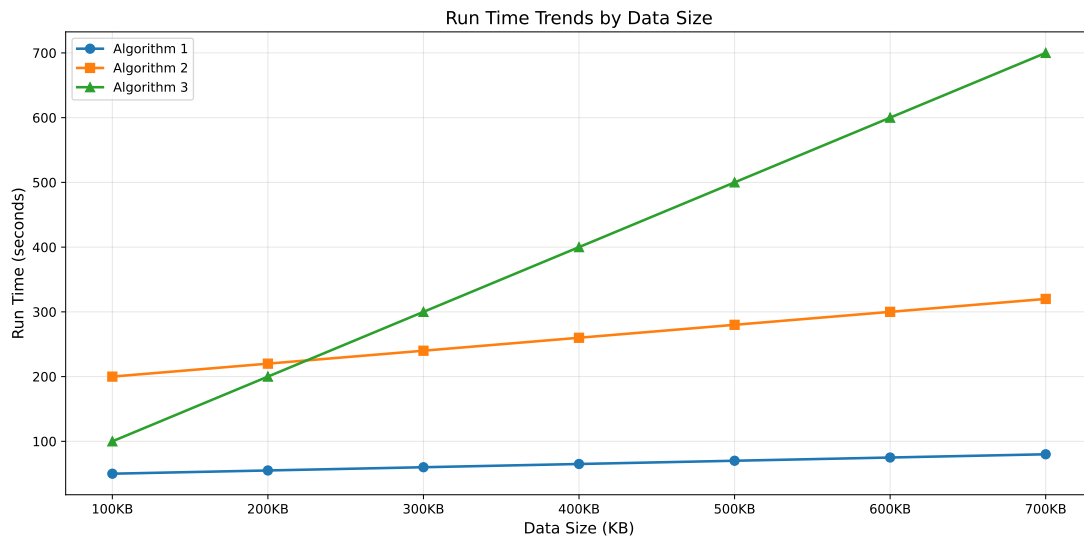


Figure 6: Algorithm Runtime Trends – Updated Dataset

The additional point continues the approximately linear empirical trend observed in the original measurements.

4.11 Updated Box Plot

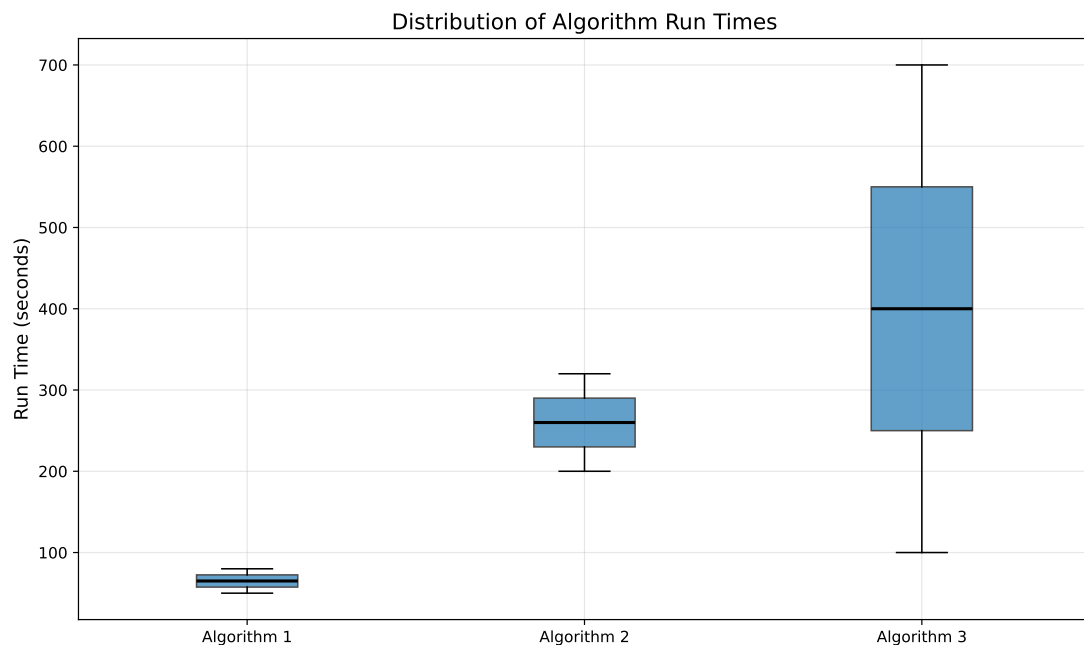


Figure 7: Runtime Distribution – Updated Dataset

The new observation increases the observed maximum and range for the algorithms, most notably for Algorithm 3.

4.12 Runtime Interpretation

The measurements indicate three distinct empirical performance levels.

Algorithm 1 is consistently the fastest of the three algorithms in the observed range. Algorithm 2 has a larger runtime but maintains a regular increase with input size. Algorithm 3 produces the largest values at the higher input sizes.

However, these observations should not be converted directly into asymptotic complexity statements. The theoretical complexity of an algorithm depends on the algorithmic operations and their scaling with input size, not merely on a small number of measured execution times.

Therefore, the appropriate conclusion from this dataset is that the algorithms demonstrate different empirical runtime trends over the tested range. A stronger complexity analysis would require repeated measurements, larger input ranges, controlled hardware conditions, and knowledge of the underlying algorithms.

5 Project 3: Numerical Integration

5.1 Problem Formulation

The third project evaluates

$$I = \int_0^1 e^{x^2} dx.$$

The integrand

$$f(x) = e^{x^2}$$

is smooth on $[0, 1]$, but it does not possess an elementary antiderivative. Numerical quadrature is therefore a natural computational approach.

The following methods are compared:

1. Composite trapezoidal rule.
2. Composite Simpson's rule.
3. Gauss–Legendre quadrature.

The comparison emphasizes convergence and absolute error.

5.2 Reference Value

Because no elementary antiderivative is available, the project uses a high-accuracy numerical value computed using `scipy.integrate.quad` as a reference. The SciPy documentation describes `quad` as a definite-integration routine based on the QUADPACK library [5, 6].

The reference value obtained is

$$I_{\text{ref}} \approx 1.462651745907181$$

This quantity is a high-accuracy numerical reference and is not claimed to be an elementary exact analytical value.

5.3 Composite Trapezoidal Rule

For n equally spaced subintervals,

$$h = \frac{b - a}{n}.$$

The composite trapezoidal approximation is

$$T_n = \frac{h}{2} \left[f(a) + f(b) + 2 \sum_{i=1}^{n-1} f(x_i) \right].$$

For sufficiently smooth functions, the global truncation error is of order

$$O(h^2)$$

[1, 2].

5.4 Composite Simpson's Rule

For even n ,

$$S_n = \frac{h}{3} \left[f(a) + f(b) + 4 \sum_{\substack{i=1 \\ i \text{ odd}}}^{n-1} f(x_i) + 2 \sum_{\substack{i=2 \\ i \text{ even}}}^{n-2} f(x_i) \right].$$

For sufficiently smooth functions, the global truncation error is of order

$$O(h^4),$$

which explains the faster convergence observed in the experiment [1, 2].

5.5 Gauss–Legendre Quadrature

Gaussian quadrature approximates an integral by selecting nodes and weights that provide high-order accuracy.

After mapping the standard interval $[-1, 1]$ to $[a, b]$,

$$x = \frac{b-a}{2}t + \frac{a+b}{2},$$

the approximation is

$$I \approx \frac{b-a}{2} \sum_{i=1}^n w_i f\left(\frac{b-a}{2}t_i + \frac{a+b}{2}\right).$$

For the implementation, Gauss–Legendre nodes and weights are used. The method is particularly effective for smooth integrands [7, 2].

5.6 Trapezoidal and Simpson Results

The numerical results for increasing numbers of subintervals are shown in Table 7.

Table 7: Convergence of Trapezoidal and Simpson Methods

n	Trapezoidal	Simpson
2	1.5715831655	1.4757305825
4	1.4906788617	1.4637107604
8	1.4697122764	1.4627234147
16	1.4644203101	1.4626563214
32	1.4630941026	1.4626520334
64	1.4627623486	1.4626517639
128	1.4626793974	1.4626517470
256	1.4626586588	1.4626517460
512	1.4626534741	1.4626517459
1024	1.4626521780	1.4626517459

At $n = 1024$,

$$T_{1024} \approx 1.4626521780$$

and

$$S_{1024} \approx 1.4626517459.$$

Using

$$E_n = |I_n - I_{\text{ref}}|,$$

the approximate errors are

$$E_T \approx 4.32 \times 10^{-7},$$

and

$$E_S \approx 4.32 \times 10^{-10}.$$

Thus, for this problem and this subdivision level, Simpson's method is approximately three orders of magnitude more accurate than the trapezoidal approximation.

5.7 Gaussian Quadrature Results

Table 8: *Convergence of Gauss–Legendre Quadrature*

Number of Points	Approximation	Absolute Error
2	1.454167889239	8.48×10^{-3}
3	1.462409711477	2.42×10^{-4}
4	1.462646815657	4.93×10^{-6}
5	1.462651668019	7.79×10^{-8}
6	1.462651744904	1.00×10^{-9}
8	1.462651745907	1.02×10^{-13}
10	1.462651745907	4.44×10^{-16}

The error decreases extremely rapidly as the number of quadrature points increases.

5.8 Graphical Convergence Analysis

The generated convergence comparison is shown in Figure 8.

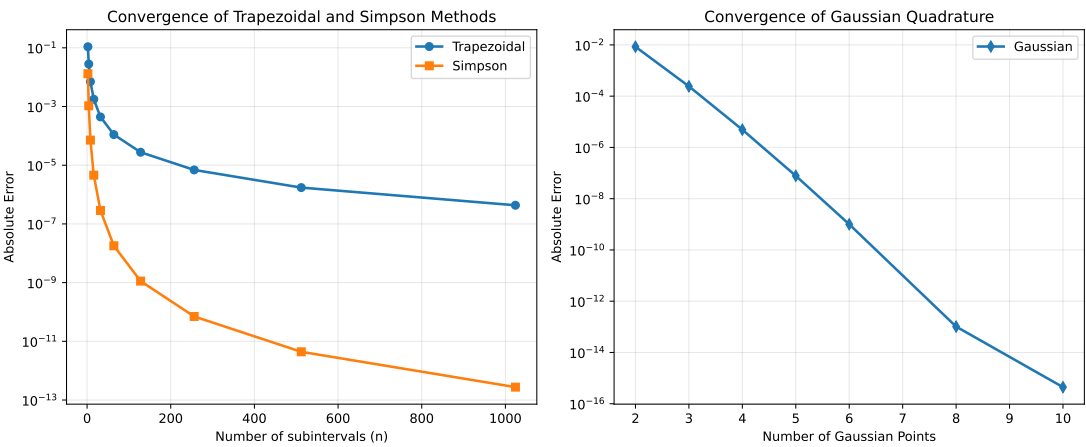


Figure 8: Numerical Integration Comparison and Convergence

The graphical output complements the numerical tables by illustrating how the approximations approach the reference value.

5.9 Method Comparison

Table 9: Summary of Numerical Integration Methods

Method	Primary Strength	Main Consideration
Trapezoidal Rule	Simple and robust implementation	Second-order convergence for smooth functions
Simpson’s Rule	Higher-order accuracy for smooth functions	Requires an even number of subintervals
Gauss–Legendre	Very high accuracy with relatively few evaluations	Requires appropriate nodes and weights

For the specific smooth function studied here, Gaussian quadrature reaches near machine precision with ten points. This does not imply that it will always dominate every other integration method; performance depends on the integrand, interval, required tolerance, and computational constraints.

6 Cross-Project Discussion

The three experiments address different mathematical tasks but illustrate several common principles of numerical computing.

6.1 Approximation and Reference Values

Interpolation provides an exact fit at known nodes but does not guarantee that the interpolating function equals an unknown underlying function between those nodes.

Numerical integration introduces a different approximation problem. The computed quadrature result is an approximation to a definite integral, and the present experiment uses a high-accuracy numerical computation as a reference.

Consequently, numerical analysis requires careful distinction between mathematical exactness and numerical accuracy.

6.2 Convergence as a Diagnostic Tool

A single numerical result does not provide enough information to assess the quality of a numerical method. Convergence experiments are more informative because they reveal how the result changes when the computational resolution is increased.

The integration experiment demonstrates this directly. The trapezoidal and Simpson approximations progressively approach the reference value, while Gaussian quadrature achieves rapid error reduction as the number of nodes increases.

6.3 Global Versus Local Approximation

The interpolation experiment illustrates the difference between a global representation and a local representation.

Lagrange interpolation produces a single global polynomial. Cubic spline interpolation instead constructs local cubic pieces joined through smoothness constraints.

This distinction is important in numerical modeling because local methods can reduce the influence of distant observations and avoid some undesirable behavior associated with high-degree global approximations.

6.4 Data Analysis as a Numerical Workflow

The Excel experiment demonstrates that practical numerical computing includes considerably more than implementing mathematical formulas.

A reliable computational workflow should also address:

- input validation,
- data-type consistency,
- reproducible transformations,
- statistical summaries,
- visualization,

- automated output generation, and
- protection against duplicate or inconsistent updates.

The automatic 700KB insertion is a small but concrete example of this principle.

7 Limitations and Future Work

7.1 Interpolation

Only six observations are available. Future experiments could compare the methods using larger datasets and different node distributions, including uniformly spaced and non-uniformly spaced nodes.

The interpolation error could also be measured against a known analytical function rather than only comparing the resulting curves.

7.2 Runtime Analysis

The runtime dataset is intentionally small and therefore cannot establish formal asymptotic complexity.

A more rigorous performance study would repeat each algorithm multiple times, use a wider range of input sizes, control system conditions, report measurement variability, and investigate the relationship between input size and runtime using appropriate statistical models.

7.3 Numerical Integration

The function e^{x^2} is smooth and well behaved on the selected interval. Additional experiments involving discontinuities, singularities, oscillatory functions, or rapidly varying functions would provide a more comprehensive assessment of the numerical integration methods.

8 Conclusion

This project implemented and evaluated numerical techniques for interpolation, computational data analysis, and numerical integration.

The interpolation experiment showed that both Lagrange interpolation and natural cubic splines can reproduce the supplied observations exactly, but their underlying constructions differ. Lagrange interpolation provides a single global polynomial, whereas cubic splines use locally defined cubic pieces connected through smoothness conditions.

The Excel analysis demonstrated a reproducible computational workflow for loading, validating, analyzing, visualizing, and updating structured data. The three visualization techniques provide complementary perspectives: bar charts emphasize direct comparison, line charts emphasize trends, and box plots summarize distributions.

The numerical integration experiment provided the clearest quantitative comparison of convergence. Simpson's rule substantially outperformed the trapezoidal rule at the tested subdivision levels, while Gauss–Legendre quadrature achieved extremely high accuracy using a small number of evaluation points for the smooth integrand under consideration.

More broadly, the project demonstrates that effective numerical computing requires a combination of mathematical theory, careful implementation, quantitative error analysis, visualization, and reproducibility. The results should therefore be interpreted not merely as numerical answers, but as evidence of the computational behavior of the implemented methods under clearly specified experimental conditions.

9 Source Code and Repository

The complete implementation, input data, generated outputs, and project documentation are maintained in the following GitHub repository:

github.com/AradCharon/numerical-methods-toolkit

The repository contains the three project implementations together with their generated numerical results and visualization files.

References

- [1] R. L. Burden and J. D. Faires, *Numerical Analysis*, 9th ed. Boston, MA: Cengage Learning, 2010.
- [2] S. C. Chapra and R. P. Canale, *Numerical Methods for Engineers*, 7th ed. New York, NY: McGraw-Hill Education, 2015.
- [3] The pandas Development Team, “pandas.read_excel documentation,” https://pandas.pydata.org/docs/reference/api/pandas.read_excel.html, 2026, accessed September 14, 2026.
- [4] The Matplotlib Development Team, “matplotlib.pyplot.savefig documentation,” https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.savefig.html, 2026, accessed September 14, 2026.
- [5] SciPy Developers, “scipy.integrate.quad documentation,” <https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.quad.html>, 2026, accessed September 14, 2026.
- [6] R. Piessens, E. de Doncker-Kapenga, C. W. Ueberhuber, and D. Kahaner, *QUADPACK: A Subroutine Package for Automatic Integration*, ser. Springer Series in Computational Mathematics. Berlin, Germany: Springer-Verlag, 1983, vol. 1.
- [7] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3rd ed. Cambridge, UK: Cambridge University Press, 2007.